Chapter: 07 ~~Undecidability~~
Computational Complexity Theory

2023-Spring

6. b> IS P=NP? Explain. Also differentiate between Tractable and Intractable problems with examples.

Sol^n

→ No one has proven that they are equal and no one has ever proven they are not equal. So, this question still remains unanswered.

→ No one has ever found a deterministic polynomial time algorithm for any of these problem ~~so we can sa~~ so that we can show P=NP.

→ On the other hand, no one has ever succeeded in proving that no deterministic polynomial time exists, either for class NP problem. ~~However~~.

⇒ Therefore, we can't ~~say~~ also say P≠NP. However, many researcher believe that N< >NP but no one knows for sure.

	Tractable Problems	Intractable Problems
i>	~~The~~ Tractable problem is solved by a polynomial time algorithm. i.e. $O(n^2)$, $O(n^3)$, ~~etc~~ etc	Intractable problem is not solved by a polynomial time algorithm. i.e. $O(2^n)$, $O(n^n)$, etc.
ii>	In tractable problem, the upper bound is polynomial.	In intractable problem, the lower bound is exponential.
iii>	Tractable problem is ~~can~~ easy.	Intractable problem is hard.
iv>	Tractable problem examples: searching an unordered list, Multiplication of integers etc.	Intractable problem examples: Tower of Hanoi, list of all permutations of n numbers etc.

2022-Fall

6.b> Explain Computational Complexity Theory. What are P, NP and NP-complete problem? Explain with examples.

Sol^n Computational Complexity Theory:

→ Computational complexity is the branch of theory of computation that deals with the ~~different~~ inherent properties of computation in the field of computer science.

→ Complexity theory focuses to measure the problems that are decidable or computable in terms of space and time.

i> Time Complexity:

→ Time complexity is a measure of how long a computation takes to execute.

→ In case of TM, time complexity is the number of moves made to perform a computation.

→ In case of digital computer, time complexity is the number of machine cycles required to compute a problem.

ii> Space Complexity:

→ Space complexity is a measure of how much space is required to compute a problem.

→ In case of TM, space complexity is the measure of tape squares used for a computation.

→ In case of digital computer, space complexity is the number of bytes required to compute a problem.

* P-Problems:

→ P-problems are the problems that can be solved in a polynomial time by deterministic turing machine

→ For example:

i> Krushal Algorithm ii> Decidable Problem

* NP-problems:

→ NP-problems are the problems that can be solved in a non-deterministic polynomial time by non-deterministic turing machine.

→ for example: Undecidable problem.

* NP-Complete Problems:

→ NP-Complete problems are the problems in which every language in NP reduces to P in polynomial time.

→ For example: Travelling salesman Problem, ~~vertex cover problem~~ etc.
circuit satisfiability problem, coververtex problem.

2019. Spring

6. a) How does computability differ from complexity theory?

SOL

S.N.	Computability theory	Complexity theory
i)	Computability theory is concerned with what can be computed v/s what cannot.	Complexity theory is concerned with how efficiently can it be computed.
ii)	Computability theory ~~consists~~ is interested in a fine-grained classification of computable problems.	Complexity theory is interested in a fine-grained classification of uncomputable problems.
iii)	~~In complexity theory, diagonalization proves the time and space hierarchy theorems.~~	~~In co~~
iii)	In computability theory, diagonalization proves the undecidability of halting problems.	In complexity theory, diagonalization proves the time and space hierarchy theorems.

Differences between NP hard and NP Complete problems.

	NP-hard Problems	NP-Complete Problems
i)	A problem X is said to be NP-hard, if there is an NP-complete problem Y, such that Y is reducible to X in polynomial time.	A problem X is said to be NP-complete, if there is an NP-problem Y, such that Y is reducible to X in polynomial time.
ii)	To solve NP-hard problem it doesnot have to be in NP problems.	To solve NP-complete problem, it must be in both NP and NP-hard problems.
iii)	In NP-hard problem, time is unknown.	In NP-complete problem, time is known.
iv)	NP-hard problem is not a decision problem.	NP-complete problem is exclusively a decision problem.
v)	All NP-hard problems are not NP-Complete.	All NP-complete problem are NP-hard.
vi)	For example: halting problem, the subset sum problem etc.	For example: Salesman problem, vertex cover problem, hamiltonian cycle problem etc.
vii)	NP-hard is the superset of NP-complete problems.	NP-complete is the subset of NP-hard problems.

chapter: 6 Undecidability

2020-~~Spring~~Fall / 2021-Fall

6. a) Write about church turing thesis and universal turing machine.

Sol

* Church Turing Thesis:

→ church turing thesis states that "No computational procedure will be considered as an algorithm unless it can be represented as a Turing Machine."

→ In simple words, "A Turing machine can compute anything that can be computed by a general purpose digital computer.

→ This statement is called church Turing thesis because Alonzo ~~chruch~~ church (1936), gave many sophisticated reasons for believing it church's original statement was slightly different from this thesis because his thesis was presented slightly before the Turing invented his machines.

→ Church actually said that any machine that can do all list of operations will be able to perform all ~~convei~~ conceivable algorithms.

* Universal Turing Machines:

→ Universal Turing Machine is a turing machine that can simulate an arbitrary Turing machine on arbitrary input.

→ The universal turing machine essentially achieve this by reading both description of machine to be simulated as well as the input thereof from its tape.

→ In other words, universal Turing machine 'U' takes two arguments, a description of a machine Tm, "Tm" and a description of an input string w, "w".

→ The functional notation of turing machine is given by

$$U(m, w) = m(w).$$

2021-Spring

6.a. Illustrate your understanding of Halting problems.

→ Halting problem is one of the most well-known problems proven to be undecidable.

→ The Halting problem is a problem that determines whether a computer program will eventually stop or run forever.

→ The Halting problem is an unsolvable problem. There is no program that can solve the halting problem for general enough computer programs.

→ The Halting problem is important for understanding how and why some problems are undecidable.

→ The Halting problem is undecidable because no algorithm or program can accurately determine for all cases whether another program will eventually stop or run forever.

→ The example of Halting problem is blank tape Halting problem.

2022-Fall

6.a> Compare and contrast the relationship of Recursive and Recursively Enumerable Language.

Soln

	Recursive Language	Recursively Enumerable Language
i>	In recursive language, the TM accepts all valid strings that are part of the language and rejects all the strings that are not part of a given language and halt.	In recursively enumerable language, the TM accepts all valid strings that are part of the language and rejects all the strings that are not part of a given language but do not halt and starts an infinite loop.
ii>	Recursive language is also known Turing decidable language.	Recursively enumerable language is also known as Turing recognizable language.
iii>	Recursive language accepts only finite loop.	Recursively enumerable language accepts in finte loops.
iv>	Recursive language is also known as context sensitive language.	Recursively enumerable language is also known as RE languages or type-0 language.
v>	Recursive language has two states: • Halt and Accept • Halt and Reject	Recursively enumerable language has 3 states: • Halt and Accept • Halt and Reject • Never Halt.

2019-spring

5.a> What is recursive and recursively enumerable language? show that the union of two recursive language is also reaursive.

sol

* Recursive Language:

→ Recursive Language is the language in which the TM accepts all valid strings that are part of the language and rejects all the strings that are not part of a given language and halt.

* Recursively Enumerable Language:

→ Recursively Enumerable Language is a language in which the TM accepts all valid strings that are part of the language and rejects all the strings are not part of the given language but do not halt and starts an infinite loop.

To prove: The union of two recursive languages is recursive

Proof:

→ Let L_1 and L_2 be the two recursive languages accepted by turing machines Tm_1 and Tm_2.

→ We construct a Turing machine Tm, which first simulates Tm_1.

→ If Tm_1 accepts, then Tm also accepts.

→ If Tm_1 rejects then Tm simulates Tm_2 and Tm accepts if and only if Tm_2 accepts

→ Since both Tm_1 and Tm_2 are algorithm so Tm is gauranteed to halt.

→ Clearly Tm accepts $L_1 \cup L_2$.

→ So, the union of two recursive language, $L_1 \cup L_2$ is also recursive.

To prove: The complement of a recursive language is also recursive;

Proof:

→ Let L be a recursive language accepted by Turing machine Tm.

→ We construct a Turing machine Tm' from Tm so that if Tm enters a final state on input w, then Tm' halts without accepting.

→ If Tm halts without accepting Tm' enters a final state.

→ Since one of these two events occurs, Tm' is an algorithm.

→ Clearly Tm' accepts L.

→ Thus, the complement of a recursive language L is also recursive.

2016-spring

5.a> Explain the properties of recursive and recursively enumerable languages.

Sol
The properties of recursive and recursively enumerable languages are as follows:

i> The complement of a recursive language is also recursive.

→ Let L_1 be a recursive language accepted by Turing machine Tm.

→ We construct a Turing machine Tm' from Tm, that accepts L.

ii> The union of two recursive language is also recursive.

→ Let L_1 and L_2 be the two recursive language accepted by Turing machine Tm_1 and Tm_2.

→ We construct a Turing machine Tm, that accepts $L_1 \cup L_2$.

iii> The union of two recursively enumerable language is also recursively enumerable.

→ Let L_1 and L_2 be two recursively enumerable languages accepted by Turing machine Tm_1 and Tm_2

→ we construct a Turing machine Tm, that accepts $L_1 \cup L_2$.

V) If a language L and its complement $\bar{L}$ is recursive.

→ Let L be a language and $\bar{L}$ be its complement accepted by Turing machine Tm_1 and Tm_2 respectively.

→ We construct a Turing machine Tm, that accepts $\bar{L}$.

VI) If L is a recursive language then $\Sigma^* - L$ is also recursive

→ Let L be a recursive language accepted by Turing machine Tm.

→ We construct a Turing machine Tm, that accepts $\Sigma^* - L$.

Chapter 5. Turing Machine

2023-spring

5.a> Describe the concept of an "accepting state" and a "halting state" in a Turing Machine.

Accepting state:

→ Accepting state is a specific type of state that indicates the successful completion of the Turing machine's task.

Halting state:

→ Halting state is a specific type of state that indicates the termination of the Turing machine's task.

5.b> Define a Turing Machine.

Turing machine is a 7-tuple and is defined as

$$M = \{Q, \Sigma, \Gamma, \delta, q_0, B, F\}$$

where,

Q = set of finite states
Σ = finite set of symbols "input alphabets"
Γ = finite set of symbols "tape alphabets"
δ = transition function
q_0 = initial state
B = blank symbol
F = set of final states.

2022-Fall

5. b) Turing machines are functionally stronger than Pushdown Automata. Justify. Also show TM are function computable.

Sol) Turing machines are functionally stronger than Pushdown Automata because of the following reasons:

i) Turing machine is deterministic in nature whereas PDA is non-deterministic.

ii) Turing machine can access any position on the infinite tape whereas PDA can access only top of the stack only in LIFO sequence.

iii) Turing machine is used for implementation in Neural Networks whereas PDA is used for designing the parsing phase of a compiler in syntax Analysis.

iv) Turing machine recognizes recursively enumerable language whereas PDA recognizes the context free languages.

v) Turing machine has more computation power than PDA.

vi) Turing machine can perform almost all type of computation like addition, subtraction, multiplication, search etc. whereas PDA can't perform all type of computation.

vii) Turing machine is composed of infinite single or many tape whereas PDA is composed of a single stack

Turing Machines are function computable.

→ This means that any function that can be computed by an algorithm, no matter how complex, can also be computed by a Turing machine.

→ A Turing machine consists of a tape, a head that moves along the tape and a set of states and transitions that determine how the machine behaves based on the symbols it reads and its current state.

→ With this setup, a Turing machine can simulate any algorithmic process, making it capable of computing any computional function.

2020-Fall

5.b) What is the configuration of Turing machine?

Soln The configuration of Turing machine ~~is defined~~ $M = \{Q, \Sigma, q_0, \delta\}$ is a triple $\{c, m, q\}$ where

1. $c \in \Sigma^*$ is a finite sequence of symbol from $\Sigma_\perp$
2. $m \in N$ is a number $<$ len (c) and
3. $q \in Q$

2021 - Spring

7.a) Write short notes on "Multi-tape turing machine".

Sol

→ A multi-tape turing machine is an ordinary turing machine with several tapes.

→ Each tape has its own head for reading and writing.

→ Multitape turing machine is defined as 6-tuples i.e. $M = \{Q, \Sigma, F, \delta, q_0, h\}$.

→ Multitape turing machine has m number of tapes with m tape heads.

→ The time complexity of multi tape turing machine is comparatively faster than single tape Turing machine.

→ In multitape turing machine, one tape is usualy used for input and other tapes are used for output.

→ The transition function of multitape turing machine is

$$(Q \times \Sigma^m) \rightarrow (Q \times \Sigma \times \{L, R, N\}^m)$$

where m is the number of tape.